

自动化系统定制化交互软件代码文档

1. 文档说明

本文档用于软件著作权申报中的源代码说明。除必要的结构说明外，正文以项目实际业务源代码为主，覆盖前台页面、前端脚本、后端服务和配置文件。

2. 源代码清单

序号	文件	说明
1	package.json	项目脚本配置
2	index.html	前台门户默认页面
3	starbucks.html	前台门户备用主题页面
4	admin.html	管理后台页面
5	assets/config.json	门户配置数据
6	assets/app.js	前台门户渲染与交互逻辑
7	assets/admin.js	管理后台交互逻辑
8	assets/styles.css	门户与后台样式
9	server.py	Python 版后端服务
10	server.js	Node.js 版后端服务

3. 代码编制说明

1. 前台门户由 HTML 页面提供挂载节点，assets/app.js 读取 assets/config.json 后动态渲染入口目录。
2. 管理后台由 admin.html 和 assets/admin.js 实现，通过 /api/* 接口完成登录、配置维护、图片上传和排序。
3. 后端提供 Python 与 Node.js 两套实现，均使用标准库完成静态资源服务、接口处理、会话认证、配置持久化和备份。
4. 配置文件采用 JSON 格式保存组织信息、分类信息和系统入口信息。

4. 项目源代码

4.1 package.json

说明：项目脚本配置。

```
{
  "scripts": {
    "start": "node server.js",
    "test": "node --test tests/*.test.mjs"
  }
}
```

4.2 index.html

说明：前台门户默认页面。

```
<!doctype html>
<html lang="zh-CN" data-theme="mintlify">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="自动化系统定制化交互软件">
```

```

<title>自动化系统定制化交互软件</title>
<link rel="stylesheet" href="assets/styles.css">
</head>
<body>
  <main id="portal-root" class="portal-root" aria-live="polite">
    <section class="loading-state">
      
      <p>正在加载系统入口...</p>
    </section>
  </main>
  <noscript>请启用 JavaScript 以使用自动化系统定制化交互软件。</noscript>
  <script src="assets/app.js" defer</script>
</body>
</html>

```

4.3 starbucks.html

说明：前台门户备用主题页面。

```

<!doctype html>
<html lang="zh-CN" data-theme="starbucks">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="自动化系统定制化交互软件 - Starbucks 风格">
    <title>自动化系统定制化交互软件 | Starbucks 风格</title>
    <link rel="stylesheet" href="assets/styles.css">
  </head>
  <body>
    <main id="portal-root" class="portal-root" aria-live="polite">
      <section class="loading-state">
        
        <p>正在加载系统入口...</p>
      </section>
    </main>
    <noscript>请启用 JavaScript 以使用自动化系统定制化交互软件。</noscript>
    <script src="assets/app.js" defer</script>
  </body>
</html>

```

4.4 admin.html

说明：管理后台页面。

```

<!doctype html>
<html lang="zh-CN" data-theme="mintlify">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="自动化系统定制化交互软件管理配置">
    <title>管理配置 - 自动化系统定制化交互软件</title>
    <link rel="stylesheet" href="assets/styles.css">
  </head>
  <body>
    <main id="admin-root" class="admin-root" aria-live="polite">
      <section class="loading-state">
        
        <p>正在加载管理配置...</p>
      </section>
    </main>
    <noscript>请启用 JavaScript 以使用管理配置页面。</noscript>
    <script src="assets/admin.js" defer</script>
  </body>
</html>

```

4.5 assets/config.json

说明：门户配置数据。

```
{
  "organization": {
    "name": "中国医学科学院医学生物学研究所",
    "portalName": "自动化系统定制化交互软件",
    "subtitle": "统一入口，快速访问常用业务系统",
    "description": "面向所内办公、业务办理、数据查询和管理工具的集中导航页面。",
    "support": "信息化管理部门"
  },
  "metrics": [
    { "label": "系统入口", "value": "12" },
    { "label": "服务分类", "value": "4" },
    { "label": "访问环境", "value": "内网" }
  ],
  "categories": [
    { "id": "all", "name": "全部入口" },
    { "id": "common", "name": "常用系统" },
    { "id": "business", "name": "业务办理" },
    { "id": "data", "name": "数据查询" },
    { "id": "admin", "name": "管理工具" }
  ],
  "systems": [
    {
      "id": "oa",
      "name": "办公协同系统",
      "description": "通知公告、流程审批、内部沟通等日常办公入口占位。",
      "category": "common",
      "url": "#",
      "status": "待配置",
      "icon": "layout-dashboard",
      "tags": ["办公", "审批", "内网"]
    },
    {
      "id": "asset",
      "name": "资产设备管理",
      "description": "设备台账、资产领用、维护记录等管理入口占位。",
      "category": "business",
      "url": "#",
      "status": "待配置",
      "icon": "boxes",
      "tags": ["资产", "设备", "台账"]
    },
    {
      "id": "inventory",
      "name": "计算机化系统清单",
      "description": "计算机化系统清单维护、查询和状态管理入口占位。",
      "category": "data",
      "url": "#",
      "status": "待配置",
      "icon": "database",
      "tags": ["清单", "查询", "数据"]
    },
    {
      "id": "lims",
      "name": "实验室信息系统",
      "description": "样本、检测、记录和实验室流程管理入口占位。",
      "category": "business",
      "url": "#",
      "status": "待配置",
      "icon": "flask",
      "tags": ["实验室", "样本", "检测"]
    }
  ]
}
```

```

},
{
  "id": "document",
  "name": "文档管理平台",
  "description": "制度文件、模板资料、归档文档统一检索入口占位。",
  "category": "common",
  "url": "#",
  "status": "待配置",
  "icon": "file-text",
  "tags": ["文档", "制度", "模板"]
},
{
  "id": "report",
  "name": "统计报表中心",
  "description": "业务指标、数据报表、分析看板集中查看入口占位。",
  "category": "data",
  "url": "#",
  "status": "待配置",
  "icon": "chart",
  "tags": ["报表", "统计", "看板"]
},
{
  "id": "account",
  "name": "账号权限管理",
  "description": "用户、角色、权限和访问申请管理入口占位。",
  "category": "admin",
  "url": "#",
  "status": "待配置",
  "icon": "shield",
  "tags": ["账号", "权限", "管理"]
},
{
  "id": "helpdesk",
  "name": "信息化服务台",
  "description": "系统问题反馈、运维工单、技术支持入口占位。",
  "category": "admin",
  "url": "#",
  "status": "待配置",
  "icon": "life-buoy",
  "tags": ["运维", "工单", "支持"]
},
{
  "id": "training",
  "name": "培训学习平台",
  "description": "在线培训、制度学习、考试记录入口占位。",
  "category": "common",
  "url": "#",
  "status": "待配置",
  "icon": "graduation",
  "tags": ["培训", "学习", "考试"]
},
{
  "id": "purchase",
  "name": "采购申请系统",
  "description": "采购申请、审批流转、进度查询入口占位。",
  "category": "business",
  "url": "#",
  "status": "待配置",
  "icon": "clipboard",
  "tags": ["采购", "审批", "流程"]
},
{
  "id": "quality",
  "name": "质量管理体系",
  "description": "偏差、变更、CAPA、质量记录管理入口占位。",

```

```

        "category": "business",
        "url": "#",
        "status": "待配置",
        "icon": "check-circle",
        "tags": ["质量", "记录", "流程"]
    },
    {
        "id": "meeting",
        "name": "会议预约系统",
        "description": "会议室查询、预约申请、会议安排入口占位。",
        "category": "common",
        "url": "#",
        "status": "待配置",
        "icon": "calendar",
        "tags": ["会议", "预约", "办公"]
    }
]
}

```

4.6 assets/app.js

说明：前台门户渲染与交互逻辑。

```

(function () {
    const root = document.getElementById('portal-root');
    const theme = document.documentElement.dataset.theme || 'mintlify';
    const state = {
        config: null,
        activeCategory: 'all',
        query: ''
    };

    const iconMap = {
        'layout-dashboard': ['M4 5h7v6H4z', 'M13 5h7v4h-7z', 'M13 11h7v8h-7z', 'M4 13h7v6H4z'],
        boxes: ['M12 3l7 4-7 4-7-4z', 'M5 9l7 4 7-4', 'M5 13l7 4 7-4'],
        database: ['M5 6c0-2 14-2 14 0v12c0 2-14 2-14 0z', 'M5 10c0 2 14 2 14 0', 'M5 14c0 2 14 2 14 0'],
        flask: ['M9 3h6', 'M10 3v5l-5 8c-.8 1.4.2 3 1.8 3h10.4c1.6 0 2.6-1.6 1.8-3l-5-8V3', 'M8 14h8'],
        'file-text': ['M7 3h7l4 4v14H7z', 'M14 3v5h5', 'M9 12h6', 'M9 16h6'],
        chart: ['M5 19V9', 'M12 19V5', 'M19 19v-7', 'M4 19h17'],
        shield: ['M12 3l7 3v5c0 5-3 8-7 10-4-2-7-5-7-10V6z', 'M9 12l2 2 4-4'],
        'life-buoy': ['M12 2la9 9 0 1 0 0-18 9 9 0 0 0 0 18z', 'M8 8l3 3', 'M16 8l-3 3', 'M8 16l3-3', 'M16 16l-3-3'],
        graduation: ['M3 8l9-4 9 4-9 4z', 'M7 10v5c3 2 7 2 10 0v-5', 'M21 8v6'],
        clipboard: ['M9 4h6l1 2h3v15H5V6h3z', 'M9 4h6', 'M8 11h8', 'M8 15h6'],
        'check-circle': ['M12 2la9 9 0 1 0 0-18 9 9 0 0 0 0 18z', 'M8.5 12l2.5 2.5L16 9'],
        calendar: ['M6 5h12v15H6z', 'M8 3v4', 'M16 3v4', 'M6 10h12']
    };

    function escapeHtml(value) {
        return String(value)
            .replace(/&/g, '&amp;')
            .replace(/</g, '&lt;')
            .replace(/>/g, '&gt;')
            .replace(/"/g, '&quot;')
            .replace(/'/g, '&#039;');
    }

    function iconSvg(name) {
        const paths = iconMap[name] || iconMap['layout-dashboard'];
        const content = paths.map((path) => `<path d="${path}"></path>`).join('');
        return `<svg class="system-icon-svg" viewBox="0 0 24 24" aria-hidden="true">${content}</svg>`;
    }

    function getCategoryName(id) {
        const category = state.config.categories.find((item) => item.id === id);
        return category ? category.name : '未分类';
    }

```

```

    }

    function filterSystems() {
      const query = state.query.trim().toLowerCase();
      return state.config.systems.filter((system) => {
        const matchesCategory = state.activeCategory === 'all' || system.category === state.activeCa
        tegrity;
        const searchText = [
          system.name,
          system.description,
          system.status,
          getCategoryName(system.category),
          ...(system.tags || [])
        ].join(' ').toLowerCase();
        return matchesCategory && (!query || searchText.includes(query));
      });
    }

    function categoryCounts() {
      return state.config.systems.reduce((counts, system) => {
        counts.all += 1;
        counts[system.category] = (counts[system.category] || 0) + 1;
        return counts;
      }, { all: 0 });
    }

    function activeCategoryName() {
      return getCategoryName(state.activeCategory);
    }

    function statusTone(status) {
      if (status === '可访问') return 'is-ready';
      if (status === '维护中') return 'is-maintenance';
      return 'is-pending';
    }

    function renderCategories() {
      const categories = state.config.categories;
      const counts = categoryCounts();
      return `
        <div class="category-bar" role="tablist" aria-label="系统分类">
          ${categories.map((category) => `
            <button
              class="category-pill ${state.activeCategory === category.id ? 'is-active' : ''}"
              type="button"
              role="tab"
              aria-selected="${state.activeCategory === category.id}"
              data-category="${escapeHtml(category.id)}"
            >
              <span>${escapeHtml(category.name)}</span>
              <strong>${escapeHtml(counts[category.id] || 0)}</strong>
            </button>
          `).join('')}
        </div>
      `;
    }

    function renderSystems(systems) {
      if (!systems.length) {
        return `
          <section class="empty-state" aria-live="polite">
            <div class="empty-state-mark">${iconSvg('database')}</div>
            <h2>没有找到匹配的系统入口</h2>
            <p>请调整关键词或切换分类。真实系统清单接入后，这里会自动展示对应入口。</p>
            <button class="button button-secondary" type="button" data-reset-search>重置筛选</butto
n>
          </section>
        `;
      }
    }

```

```

    `;
  }

  return `
    <section class="system-grid" aria-label="系统入口列表">
      ${systems.map((system) => {
        const disabled = system.url === '#';
        const tags = (system.tags || []).slice(0, 3);
        const hasImage = Boolean(system.image);
        const cardClass = hasImage ? 'system-card system-card-layout has-image' : 'system-card';
        const cardTag = disabled ? 'article' : 'a';
        const cardAttributes = disabled
          ? ''
          : ` href="${escapeHtml(system.url)}" target="_blank" rel="noopener noreferrer"`;
        const imageMarkup = hasImage
          ? `<div class="system-card-media"></div>`
          : '';

        return `
          <${cardTag} class="${cardClass} ${disabled ? 'is-placeholder' : 'system-card-link'}"
            ${cardAttributes}>
            <div class="system-card-content">
              <div class="system-card-top">
                <span class="system-category">${escapeHtml(getCategoryName(system.category))}</s
pan>
                <span class="status-badge ${statusTone(system.status)}">${escapeHtml(system.stat
us || '可访问')}</span>
              </div>
              <div class="system-card-body">
                <div class="system-title-line">
                  <div class="system-icon">${iconSvg(system.icon)}</div>
                  <h2>${escapeHtml(system.name)}</h2>
                </div>
                <p>${escapeHtml(system.description)}</p>
              </div>
              <div class="tag-row">
                ${tags.map((tag) => `<span>${escapeHtml(tag)}</span>`).join('')}
              </div>
              <div class="system-card-footer">
                <span>${disabled ? '地址待配置' : '新窗口打开'}</span>
                <span aria-hidden="true">${disabled ? '待' : '↗'}</span>
              </div>
            </div>
            ${imageMarkup}
          </${cardTag}>
        `;
      }).join('')}
    </section>
  `;
}

function renderMetrics() {
  const systemCount = state.config.systems.length;
  const categoryCount = state.config.categories.filter((item) => item.id !== 'all').length;
  const envMetric = state.config.metrics.find((metric) => metric.label === '访问环境');
  const metrics = [
    { label: '系统入口', value: String(systemCount) },
    { label: '服务分类', value: String(categoryCount) },
    { label: '访问环境', value: envMetric?.value || '内网' }
  ];
}

return `
  <div class="metric-row" aria-label="门户概览">
    ${metrics.map((metric) => `
      <div class="metric-item">

```

```

        <strong>${escapeHtml(metric.value)}</strong>
        <span>${escapeHtml(metric.label)}</span>
      </div>
    `).join('')}
  </div>
  `;
}

function renderCategorySummary() {
  const counts = categoryCounts();
  return `
    <div class="category-summary" aria-label="分类概览">
      ${state.config.categories.filter((category) => category.id !== 'all').map((category) => `
        <button type="button" data-category="${escapeHtml(category.id)}" class="category-summary
- item ${state.activeCategory === category.id ? 'is-active' : ''}">
          <span>${escapeHtml(category.name)}</span>
          <strong>${escapeHtml(counts[category.id] || 0)}</strong>
        </button>
      `).join('')}
    </div>
  `;
}

function renderPortal() {
  const config = state.config;
  const systems = filterSystems();
  root.innerHTML = `
    <header class="top-nav">
      <a class="brand" href="${theme === 'starbucks' ? 'starbucks.html' : 'index.html'}" aria-la
bel="${escapeHtml(config.organization.portalName)}">首页</a>
      
      <span>
        <strong>${escapeHtml(config.organization.name)}</strong>
        <small>${escapeHtml(config.organization.portalName)}</small>
      </span>
    </a>
    <nav class="nav-actions" aria-label="主题与帮助">
      <a class="button button-secondary" href="admin.html">管理配置</a>
      <a class="button button-primary" href="#systems">入口目录</a>
    </nav>
  </header>

    <section class="hero-section">
      <div class="hero-copy">
        <span class="eyebrow">Institute service index</span>
        <h1>${escapeHtml(config.organization.portalName)}</h1>
        <p>${escapeHtml(config.organization.description || config.organization.subtitle)}</p>
        <div class="hero-actions">
          <label class="search-shell">
            <span class="sr-only">搜索系统入口</span>
            <svg viewBox="0 0 24 24" aria-hidden="true"><path d="M10.5 18a7.5 7.5 0 1 1 5.3-2.2L
21 21"></path></svg>
            <input id="portal-search" type="search" placeholder="搜索系统名称、标签或说明" value="
${escapeHtml(state.query)}" autocomplete="off">
          </label>
          <button class="button button-primary" type="button" data-focus-search>快速搜索</butto
n>
        </div>
      </div>
      ${renderMetrics()}
    </div>
    <div class="hero-panel" aria-label="门户配置说明">
      <div class="panel-kicker">入口清单</div>
      <h2 id="active-category-name">${escapeHtml(activeCategoryName())}</h2>
      <p>系统入口、分类和状态由 <code>assets/config.json</code> 集中维护，管理后台保存后前台刷新即
生效。</p>
      ${renderCategorySummary()}
  `;
}

```



```

    </div>
  </section>

  <div class="portal-divider" role="presentation">
    <div class="portal-divider-line"></div>
    <div class="portal-divider-line"></div>
  </div>

  <section id="systems" class="systems-section">
    <div class="catalog-heading">
      <div>
        <span class="eyebrow">Service catalogue</span>
        <h2>系统入口目录</h2>
      </div>
      <p>当前显示 <strong id="result-count">${escapeHtml(systems.length)}</strong> 个入口。可按
分类筛选，或搜索系统名称、说明和标签。</p>
    </div>
    ${renderCategories()}
    <div id="system-results">${renderSystems(systems)}</div>
  </section>

  <footer class="site-footer">
    <div>
      <strong>${escapeHtml(config.organization.name)}</strong>
      <p>${escapeHtml(config.organization.support)}维护。静态部署版本，系统清单通过配置文件更新。<
/p>
    </div>
    <a href="#portal-root">返回顶部</a>
  </footer>
`;

bindEvents();
}

function bindEvents() {
  const search = document.getElementById('portal-search');
  if (search) {
    search.addEventListener('input', (event) => {
      state.query = event.target.value;
      updateResults();
    });
  }

  document.querySelectorAll('[data-category]').forEach((button) => {
    button.addEventListener('click', () => {
      state.activeCategory = button.dataset.category;
      updateResults();
    });
  });

  document.querySelectorAll('[data-focus-search]').forEach((button) => {
    button.addEventListener('click', () => {
      document.getElementById('portal-search')?.focus();
    });
  });

  document.querySelectorAll('[data-reset-search]').forEach((button) => {
    button.addEventListener('click', () => {
      state.query = '';
      state.activeCategory = 'all';
      renderPortal();
    });
  });
}

function updateResults() {
  const results = document.getElementById('system-results');

```

```

    if (!results) return;
    const systems = filterSystems();
    results.innerHTML = renderSystems(systems);
    const resultCount = document.getElementById('result-count');
    if (resultCount) {
        resultCount.textContent = String(systems.length);
    }
    const activeCategory = document.getElementById('active-category-name');
    if (activeCategory) {
        activeCategory.textContent = activeCategoryName();
    }
    const categories = document.querySelector('.category-bar');
    if (categories) {
        categories.outerHTML = renderCategories();
    }
    const summary = document.querySelector('.category-summary');
    if (summary) {
        summary.outerHTML = renderCategorySummary();
    }
    bindEvents();
}

function renderError(error) {
    root.innerHTML = `
        <section class="empty-state load-error">
            <div class="empty-state-mark">${iconSvg('life-buoy')}</div>
            <h1>配置加载失败</h1>
            <p>请检查 <code>assets/config.json</code> 是否存在且格式正确。</p>
            <pre>${escapeHtml(error.message)}</pre>
        </section>
    `;
}

async function init() {
    try {
        const response = await fetch('assets/config.json', { cache: 'no-store' });
        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }
        state.config = await response.json();
        renderPortal();
    } catch (error) {
        renderError(error);
    }
}

init();
})();

```

4.7 assets/admin.js

说明：管理后台交互逻辑。

```

(function () {
    const root = document.getElementById('admin-root');
    const state = {
        config: null,
        selectedId: null,
        query: '',
        message: '',
        error: '',
        uploadMessage: '',
        saveDialogOpen: false,
        draft: null
    };

    const defaultSystem = {

```

```

    name: '',
    description: '',
    category: 'common',
    icon: 'layout-dashboard',
    image: '',
    tags: '',
    status: '待配置',
    url: '#'
  };

  const iconOptions = [
    'layout-dashboard',
    'boxes',
    'database',
    'flask',
    'file-text',
    'chart',
    'shield',
    'life-buoy',
    'graduation',
    'clipboard',
    'check-circle',
    'calendar'
  ];

  const statusOptions = ['可访问', '待配置', '维护中'];

  function escapeHtml(value) {
    return String(value || '')
      .replace(/&/g, '&')
      .replace(/</g, '<')
      .replace(/>/g, '>')
      .replace(/"/g, '"')
      .replace(/'/g, ''');
  }

  function setNotice(message, error) {
    state.message = message || '';
    state.error = error || '';
  }

  function setUploadNotice(message) {
    state.uploadMessage = message || '';
  }

  async function api(path, options) {
    const headers = { ...(options?.headers || {}) };
    const isFormData = options?.body instanceof FormData;
    if (!isFormData && !headers['content-type']) {
      headers['content-type'] = 'application/json';
    }

    const response = await fetch(path, {
      credentials: 'same-origin',
      ...options,
      headers
    });
    const data = await response.json().catch(() => ({}));
    if (!response.ok) {
      throw new Error(data.error || `HTTP ${response.status}`);
    }
    return data;
  }

  function getSelectedSystem() {
    if (!state.config || !state.selectedId) return null;
    return state.config.systems.find((system) => system.id === state.selectedId) || null;
  }

```

```

function formValue(system) {
  const source = system || defaultSystem;
  return {
    ...defaultSystem,
    ...source,
    tags: Array.isArray(source.tags) ? source.tags.join(', ') : source.tags || ''
  };
}

function currentValues() {
  if (!state.draft) {
    state.draft = formValue(getSelectedSystem());
  }
  return state.draft;
}

function resetDraft(system) {
  state.draft = formValue(system);
}

function filteredSystems() {
  const query = state.query.trim().toLowerCase();
  if (!query) return state.config.systems;
  return state.config.systems.filter((system) => {
    return [
      system.name,
      system.description,
      system.category,
      system.status,
      ...(system.tags || [])
    ].join(' ').toLowerCase().includes(query);
  });
}

function categoryName(id) {
  const category = state.config.categories.find((item) => item.id === id);
  return category ? category.name : id;
}

function selectedSystemIndex() {
  if (!state.config || !state.selectedId) return -1;
  return state.config.systems.findIndex((system) => system.id === state.selectedId);
}

function previewIconLabel(icon) {
  const parts = String(icon || 'FI').split('-');
  return parts.map((part) => part.charAt(0)).join('').slice(0, 2).toUpperCase() || 'FI';
}

function dialogMarkup() {
  if (!state.saveDialogOpen) return '';
  return `
    <div class="admin-modal" role="dialog" aria-modal="true" aria-labelledby="save-dialog-title"
">
      <div class="admin-modal-card">
        <h2 id="save-dialog-title">配置已生效</h2>
        <p>当前修改已经写入配置文件，前台入口页刷新后即可看到最新内容。</p>
        <div class="admin-modal-actions">
          <button class="button button-primary" type="button" data-close-dialog>知道了</button>
        </div>
      </div>
    </div>
  `;
}

function renderLogin() {

```

```

root.innerHTML = `
  <section class="admin-login-shell">
    <div class="admin-login-card">
      <a class="brand" href="index.html" aria-label="返回门户首页">
        
        <span>
          <strong>自动化系统定制化交互软件</strong>
          <small>管理员配置入口</small>
        </span>
      </a>
      <div class="section-heading">
        <span class="eyebrow">Admin console</span>
        <h1>管理配置</h1>
        <p>登录后维护系统入口卡片，保存后前台页面刷新即可生效。</p>
      </div>
      ${state.error ? `<p class="admin-alert is-error">${escapeHtml(state.error)}</p>` : ''}
      <form class="admin-login-form" data-login-form>
        <label>
          <span>管理员密码</span>
          <input name="password" type="password" autocomplete="current-password" required>
        </label>
        <button class="button button-primary" type="submit">登录</button>
      </form>
      <a class="admin-back-link" href="index.html">返回系统入口</a>
    </div>
  </section>
`;
root.querySelector('[data-login-form]').addEventListener('submit', login);
}

function renderList() {
  const systems = filteredSystems();
  const selectedIndex = selectedSystemIndex();
  const isSearching = Boolean(state.query.trim());
  return `
    <section class="admin-list-panel">
      <div class="admin-panel-head">
        <div>
          <span class="eyebrow">Service cards</span>
          <h2>入口卡片</h2>
        </div>
        <button class="button button-primary" type="button" data-new-system>新增入口</button>
      </div>
      <label class="admin-search">
        <span class="sr-only">搜索系统入口</span>
        <input type="search" placeholder="搜索名称、描述、分类或标签" value="${escapeHtml(state.query)}" data-admin-search>
      </label>
      <div class="admin-order-actions">
        <button class="button button-secondary" type="button" data-move-up ${selectedIndex <= 0 || isSearching ? 'disabled' : ''}>上移</button>
        <button class="button button-secondary" type="button" data-move-down ${selectedIndex === -1 || selectedIndex >= state.config.systems.length - 1 || isSearching ? 'disabled' : ''}>下移</button>
        <span class="admin-order-hint">${isSearching ? '搜索状态下不可调整顺序' : '选择卡片后可调整显示顺序'}</span>
      </div>
      <div class="admin-card-list">
        ${systems.map((system) => `
          <button class="admin-system-row ${state.selectedId === system.id ? 'is-active' : ''}"
            type="button" data-edit-id="${escapeHtml(system.id)}">
            <span>
              <strong>${escapeHtml(system.name)}</strong>
              <small>${escapeHtml(categoryName(system.category))} / ${escapeHtml(system.status | '待配置')}</small>
            </span>
          `)}
      </div>
  `
}

```

```

        <em>${escapeHtml((system.tags || []).slice(0, 2).join(' · '))}</em>
      </button>
    `).join('')}
  </div>
</section>
`;
}

function renderPreview(values) {
  const tags = String(values.tags || '').split(',').map((tag) => tag.trim()).filter(Boolean).slice(0, 3);

  return `
    <aside class="admin-preview" aria-label="卡片预览">
      <span class="eyebrow">Preview</span>
      <article class="system-card ${values.image ? 'system-card-layout has-image' : ''}">
        <div class="system-card-content">
          <div class="system-card-top">
            <div class="system-icon">${previewIconLabel(values.icon)}</div>
            <span class="status-badge">${escapeHtml(values.status || '待配置')}</span>
          </div>
          <div class="system-card-body">
            <p class="system-category">${escapeHtml(categoryName(values.category))}</p>
            <h2>${escapeHtml(values.name || '系统名称')}</h2>
            <p>${escapeHtml(values.description || '系统描述将在这里显示。')}</p>
          </div>
          <div class="tag-row">
            ${tags.map((tag) => `<span>${escapeHtml(tag)}</span>`).join('')}
          </div>
        </div>
        ${values.image ? `<div class="system-card-media"></div>` : ''}
      </article>
    </aside>
  `;
}

function renderForm() {
  const selected = getSelectedSystem();
  const values = currentValues();
  return `
    <section class="admin-form-panel">
      <div class="admin-panel-head">
        <div>
          <span class="eyebrow">Card editor</span>
          <h2>${selected ? '编辑入口' : '新增入口'}</h2>
        </div>
        ${selected ? `<button class="button button-secondary" type="button" data-cancel-edit>取消编辑</button>` : ''}
      </div>
      <form class="admin-form" data-system-form>
        <label>
          <span>系统名称</span>
          <input name="name" value="${escapeHtml(values.name)}" required>
        </label>
        <label>
          <span>描述</span>
          <textarea name="description" rows="4" required>${escapeHtml(values.description)}</text
area>
        </label>
        <div class="admin-form-grid">
          <label>
            <span>分类</span>
            <select name="category" required>

```

```

        ${state.config.categories.filter((item) => item.id !== 'all').map((category) => `
        <option value="${escapeHtml(category.id)}" ${values.category === category.id ? '
selected' : ''}>${escapeHtml(category.name)}</option>
        `).join('')}
    </select>
</label>
<label>
    <span>图标</span>
    <select name="icon">
        ${iconOptions.map((icon) => `<option value="${icon}" ${values.icon === icon ? 'sel
ected' : ''}>${icon}</option>`).join('')}
    </select>
</label>
</div>
<label>
    <span>图片</span>
    <div class="admin-upload">
        <input name="image" type="hidden" value="${escapeHtml(values.image)}">
        <label class="button button-secondary admin-upload-button">
            <input type="file" accept="image/png,image/jpeg,image/webp,image/gif" data-image-u
pload>
                上传本地图片
            </label>
            <span class="admin-upload-path">${escapeHtml(values.image || '未上传图片')}</span>
            ${values.image ? '<button class="button admin-upload-remove" type="button" data-dele
te-image>删除图片</button>' : ''}
        </div>
        ${state.uploadMessage ? `<span class="admin-upload-help">${escapeHtml(state.uploadMess
age)}</span>` : '<span class="admin-upload-help">支持 png、jpg、webp、gif，上传后自动写入 assets/upl
oads。</span>'}
    </label>
</label>
    <span>访问地址</span>
    <input name="url" value="${escapeHtml(values.url)}" placeholder="https://... 或 #">
</label>
<div class="admin-form-grid">
    <label>
        <span>状态</span>
        <select name="status">
            ${statusOptions.map((status) => `<option value="${status}" ${values.status === sta
tus ? 'selected' : ''}>${status}</option>`).join('')}
        </select>
    </label>
    <label>
        <span>标签</span>
        <input name="tags" value="${escapeHtml(values.tags)}" placeholder="办公，审批，内网">
    </label>
</div>
<div class="admin-form-actions">
    <button class="button button-primary" type="submit">${selected ? '保存修改' : '新增入口
'}</button>
    ${selected ? `<button class="button admin-danger" type="button" data-delete-system>删
除入口</button>` : ''}
</div>
</form>
    ${renderPreview(values)}
</section>
`;
}

function renderAdmin() {
    root.innerHTML = `
        <header class="top-nav">
            <a class="brand" href="index.html" aria-label="返回门户首页">
                
                <span>

```

```

        <strong>自动化系统定制化交互软件</strong>
        <small>管理员配置</small>
    </span>
</a>
<nav class="nav-actions" aria-label="管理操作">
    <a class="button button-secondary" href="index.html">查看入口</a>
    <button class="button button-primary" type="button" data-logout>退出登录</button>
</nav>
</header>
<section class="admin-hero">
    <span class="eyebrow">Admin console</span>
    <h1>系统入口配置</h1>
    <p>集中维护系统名称、描述、分类、图标、图片、标签和访问地址。</p>
</section>
<section class="admin-shell">
    ${state.message ? `<p class="admin-alert">${escapeHtml(state.message)}</p>` : ''}
    ${state.error ? `<p class="admin-alert is-error">${escapeHtml(state.error)}</p>` : ''}
    ${renderList()}
    ${renderForm()}
</section>
    ${dialogMarkup()}
`
    ;
    bindAdminEvents();
}

function syncDraftFromForm() {
    const form = root.querySelector('[data-system-form]');
    if (!form) return;
    state.draft = {
        ...defaultSystem,
        ...Object.fromEntries(new FormData(form).entries())
    };
}

function bindAdminEvents() {
    root.querySelector('[data-logout]')?.addEventListener('click', logout);
    root.querySelector('[data-close-dialog]')?.addEventListener('click', () => {
        state.saveDialogOpen = false;
        renderAdmin();
    });
    root.querySelector('[data-new-system]')?.addEventListener('click', () => {
        state.selectedId = null;
        state.saveDialogOpen = false;
        setNotice('', '');
        setUploadNotice('');
        resetDraft(null);
        renderAdmin();
    });
    root.querySelector('[data-cancel-edit]')?.addEventListener('click', () => {
        state.selectedId = null;
        state.saveDialogOpen = false;
        setUploadNotice('');
        resetDraft(null);
        renderAdmin();
    });
    root.querySelector('[data-admin-search]')?.addEventListener('input', (event) => {
        state.query = event.target.value;
        renderAdmin();
        root.querySelector('[data-admin-search]')?.focus();
    });
    root.querySelector('[data-move-up]')?.addEventListener('click', () => moveSystem(-1));
    root.querySelector('[data-move-down]')?.addEventListener('click', () => moveSystem(1));
    root.querySelectorAll('[data-edit-id]').forEach((button) => {
        button.addEventListener('click', () => {
            state.selectedId = button.dataset.editId;
            state.saveDialogOpen = false;
            setNotice('', '');
        });
    });
}

```



```

        setUploadNotice('');
        resetDraft(getSelectedSystem());
        renderAdmin();
    });
});
root.querySelector('[data-system-form]')?.addEventListener('submit', saveSystem);
root.querySelector('[data-delete-system]')?.addEventListener('click', deleteSystem);
root.querySelector('[data-image-upload]')?.addEventListener('change', uploadImage);
root.querySelector('[data-delete-image]')?.addEventListener('click', deleteImage);
root.querySelector('[data-system-form]')?.addEventListener('input', () => {
    syncDraftFromForm();
    const preview = root.querySelector('.admin-preview');
    if (preview) preview.outerHTML = renderPreview(currentValues());
});
}

async function login(event) {
    event.preventDefault();
    const password = new FormData(event.currentTarget).get('password');
    try {
        await api('/api/login', {
            method: 'POST',
            body: JSON.stringify({ password })
        });
        setNotice('', '');
        await loadConfig();
    } catch (error) {
        setNotice('', error.message);
        renderLogin();
    }
}

async function logout() {
    await api('/api/logout', { method: 'POST', body: '{}' }).catch(() => {});
    state.config = null;
    state.selectedId = null;
    state.draft = null;
    state.saveDialogOpen = false;
    setNotice('', '');
    setUploadNotice('');
    renderLogin();
}

async function loadConfig() {
    state.config = await api('/api/config');
    if (state.selectedId && !getSelectedSystem()) {
        state.selectedId = null;
    }
    resetDraft(getSelectedSystem());
    renderAdmin();
}

async function saveSystem(event) {
    event.preventDefault();
    syncDraftFromForm();
    const body = currentValues();
    try {
        const selected = getSelectedSystem();
        const result = await api(selected ? `/api/systems/${encodeURIComponent(selected.id)}` : '/api/systems', {
            method: selected ? 'PUT' : 'POST',
            body: JSON.stringify(body)
        });
        state.config = result.config;
        state.selectedId = result.system.id;
        resetDraft(result.system);
        state.saveDialogOpen = true;
        setNotice('配置已保存, 前台刷新后生效.', '');
    }
}

```

```

    renderAdmin();
  } catch (error) {
    setNotice('', error.message);
    renderAdmin();
  }
}

async function uploadImage(event) {
  const file = event.currentTarget.files && event.currentTarget.files[0];
  if (!file) return;

  syncDraftFromForm();
  const form = new FormData();
  form.set('image', file);
  setUploadNotice('正在上传图片...');
  renderAdmin();

  try {
    const data = await api('/api/upload', {
      method: 'POST',
      body: form
    });
    state.draft.image = data.path;
    setUploadNotice(`已上传: ${data.path}`);
    renderAdmin();
  } catch (error) {
    setUploadNotice(error.message);
    renderAdmin();
  }
}

async function deleteSystem() {
  const selected = getSelectedSystem();
  if (!selected) return;
  if (!window.confirm(`确认删除"${selected.name}"?`)) return;

  try {
    const result = await api(`/api/systems/${encodeURIComponent(selected.id)}`, {
      method: 'DELETE',
      body: '{}'
    });
    state.config = result.config;
    state.selectedId = null;
    state.saveDialogOpen = false;
    setUploadNotice('');
    setNotice('入口已删除, 前台刷新后生效.', '');
    resetDraft(null);
    renderAdmin();
  } catch (error) {
    setNotice('', error.message);
    renderAdmin();
  }
}

async function deleteImage() {
  syncDraftFromForm();
  const currentPath = currentValues().image;
  if (!currentPath) return;

  try {
    await api('/api/upload', {
      method: 'DELETE',
      body: JSON.stringify({ path: currentPath })
    });
    state.draft.image = '';
    setUploadNotice('图片已删除');
    renderAdmin();
  }
}

```

```

    } catch (error) {
      setUploadNotice(error.message);
      renderAdmin();
    }
  }

  async function moveSystem(direction) {
    if (!state.config || state.query.trim()) return;
    const index = selectedSystemIndex();
    if (index === -1) return;
    const nextIndex = index + direction;
    if (nextIndex < 0 || nextIndex >= state.config.systems.length) return;

    const systems = state.config.systems.slice();
    const [moved] = systems.splice(index, 1);
    systems.splice(nextIndex, 0, moved);

    try {
      const result = await api('/api/config', {
        method: 'PUT',
        body: JSON.stringify({ systems })
      });
      state.config = result.config;
      state.selectedId = moved.id;
      resetDraft(getSelectedSystem());
      setNotice('显示顺序已更新。', '');
      renderAdmin();
    } catch (error) {
      setNotice('', error.message);
      renderAdmin();
    }
  }

  async function init() {
    try {
      const session = await api('/api/session');
      if (!session.authenticated) {
        renderLogin();
        return;
      }
      await loadConfig();
    } catch (error) {
      setNotice('', error.message);
      renderLogin();
    }
  }

  init();
})();

```

4.8 assets/styles.css

说明：门户与后台样式。

```

:root {
  --brand-blue: #003888;
  --brand-green: #009838;
  --ink: #102033;
  --ink-strong: #061426;
  --muted: #657385;
  --muted-2: #8793a1;
  --canvas: #f6f8fb;
  --paper: #ffffff;
  --paper-soft: #fbfcfe;
  --blue-soft: #eaf1fa;
  --green-soft: #e8f6ee;
  --line: #d9e2ec;

```

```

--line-strong: #c7d3df;
--warning: #9a6700;
--warning-soft: #fff5d6;
--danger: #b42318;
--danger-soft: #fff1f0;
--radius-sm: 6px;
--radius-md: 8px;
--radius-lg: 12px;
--shadow-panel: 0 18px 50px -42px rgba(0, 56, 136, 0.42);
--shadow-hover: 0 18px 36px -30px rgba(0, 56, 136, 0.36);
color-scheme: light;
font-family: "Microsoft YaHei", "PingFang SC", "Noto Sans CJK SC", system-ui, -apple-system, Bli
nkMacSystemFont, "Segoe UI", sans-serif;
}

* {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  min-width: 320px;
  background:
    linear-gradient(180deg, #ffffff 0, rgba(255, 255, 255, 0) 260px),
    linear-gradient(90deg, rgba(0, 56, 136, 0.035) 1px, transparent 1px),
    var(--canvas);
  background-size: auto, 32px 32px, auto;
  color: var(--ink);
}

a {
  color: inherit;
  text-decoration: none;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  cursor: pointer;
}

button:disabled {
  cursor: not-allowed;
}

code {
  padding: 2px 6px;
  color: var(--brand-blue);
  background: var(--blue-soft);
  border: 1px solid rgba(0, 56, 136, 0.08);
  border-radius: 4px;
  font-family: "SF Mono", Consolas, "Liberation Mono", monospace;
  font-size: 0.92em;
}

:focus-visible {
  outline: 3px solid rgba(0, 152, 56, 0.32);
  outline-offset: 3px;
}

```

```

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

.portal-root,
.admin-root {
  min-height: 100vh;
  overflow-x: hidden;
}

.top-nav {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  justify-content: space-between;
  min-height: 72px;
  gap: 20px;
  padding: 10px clamp(16px, 4vw, 56px);
  background: rgba(255, 255, 255, 0.94);
  border-bottom: 1px solid var(--line);
  backdrop-filter: blur(18px);
}

.top-nav::before {
  position: absolute;
  top: 0;
  right: 0;
  left: 0;
  height: 3px;
  content: "";
  background: linear-gradient(90deg, var(--brand-blue), var(--brand-green));
}

.brand {
  display: flex;
  align-items: center;
  min-width: 0;
  gap: 12px;
}

.brand img,
.loading-state img {
  width: 44px;
  height: 44px;
  object-fit: contain;
  background: #fff;
  border: 1px solid rgba(0, 56, 136, 0.16);
  border-radius: 50%;
}

.brand span {
  display: grid;
  min-width: 0;
  gap: 2px;
}

.brand strong {

```

```

    overflow: hidden;
    color: var(--brand-blue);
    font-size: 15px;
    font-weight: 650;
    line-height: 1.28;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.brand small {
    color: var(--muted);
    font-size: 12px;
    line-height: 1.35;
}

.nav-actions,
.hero-actions,
.metric-row,
.category-bar,
.tag-row,
.admin-order-actions,
.admin-form-actions,
.admin-upload,
.admin-modal-actions {
    display: flex;
    align-items: center;
}

.nav-actions {
    justify-content: flex-end;
    flex-wrap: wrap;
    gap: 8px;
}

.button {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-height: 40px;
    padding: 8px 15px;
    color: var(--ink);
    background: var(--paper);
    border: 1px solid var(--line-strong);
    border-radius: var(--radius-sm);
    font-size: 13px;
    font-weight: 650;
    line-height: 1.2;
    transition: transform 120ms ease, background-color 120ms ease, border-color 120ms ease, color 120ms ease, box-shadow 120ms ease;
}

.button:hover {
    border-color: rgba(0, 56, 136, 0.45);
    box-shadow: var(--shadow-hover);
}

.button:active {
    transform: translateY(1px);
}

.button-primary {
    color: #fff;
    background: var(--brand-blue);
    border-color: var(--brand-blue);
}

.button-primary:hover {
    background: #052f70;
}

```

```

    border-color: #052f70;
}

.button-secondary {
    color: var(--brand-blue);
    background: #fff;
}

.eyebrow,
.panel-kicker {
    display: inline-flex;
    align-items: center;
    color: var(--brand-green);
    font-size: 11px;
    font-weight: 750;
    letter-spacing: 0.08em;
    text-transform: uppercase;
}

.hero-section {
    position: relative;
    display: grid;
    grid-template-columns: minmax(0, 1.1fr) minmax(320px, 0.62fr);
    gap: clamp(18px, 3vw, 32px);
    padding: clamp(36px, 6vw, 72px) clamp(16px, 4vw, 56px) clamp(28px, 4vw, 44px);
    background:
        linear-gradient(90deg, rgba(234, 241, 250, 0.92), rgba(255, 255, 255, 0.7) 54%, rgba(232, 246,
        238, 0.76)),
        #fff;
    border-bottom: 1px solid var(--hero-bottom-border, var(--line));
}

.hero-section::after {
    position: absolute;
    right: clamp(16px, 4vw, 56px);
    bottom: 0;
    left: clamp(16px, 4vw, 56px);
    height: 1px;
    content: "";
    background: linear-gradient(90deg, var(--brand-blue), var(--brand-green), transparent);
    opacity: 0.45;
}

.hero-copy {
    position: relative;
    z-index: 1;
    max-width: 760px;
}

.hero-copy h1 {
    max-width: 720px;
    margin: 8px 0 0;
    color: var(--ink-strong);
    font-size: clamp(34px, 5vw, 58px);
    font-weight: 650;
    line-height: 1.08;
    letter-spacing: 0;
}

.hero-copy p {
    max-width: 640px;
    margin: 16px 0 0;
    color: var(--muted);
    font-size: clamp(15px, 1.5vw, 18px);
    line-height: 1.75;
}

.hero-actions {

```

```

    flex-wrap: wrap;
    gap: 10px;
    margin-top: 24px;
}

.search-shell {
    flex: 1 1 320px;
    max-width: 520px;
    min-height: 48px;
    gap: 10px;
    padding: 0 14px;
    color: var(--muted);
    background: rgba(255, 255, 255, 0.94);
    border: 1px solid var(--line-strong);
    border-radius: var(--radius-md);
    box-shadow: 0 18px 44px -38px rgba(0, 56, 136, 0.55);
}

.search-shell svg {
    width: 18px;
    height: 18px;
    flex: 0 0 auto;
    fill: none;
    stroke: currentColor;
    stroke-width: 2;
    stroke-linecap: round;
}

.search-shell input {
    width: 100%;
    min-width: 0;
    color: var(--ink);
    background: transparent;
    border: 0;
    outline: 0;
}

.search-shell input::placeholder {
    color: var(--muted-2);
}

.metric-row {
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 22px;
}

.metric-item {
    min-width: 116px;
    padding: 12px 14px;
    background: rgba(255, 255, 255, 0.78);
    border: 1px solid rgba(0, 56, 136, 0.12);
    border-radius: var(--radius-md);
}

.metric-item strong {
    display: block;
    color: var(--brand-blue);
    font-size: 24px;
    font-weight: 650;
    line-height: 1;
}

.metric-item span {
    display: block;
    margin-top: 6px;
    color: var(--muted);
    font-size: 12px;
}

```



```

}

.hero-panel {
  align-self: stretch;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  min-height: 290px;
  padding: 22px;
  background: rgba(255, 255, 255, 0.88);
  border: 1px solid rgba(0, 56, 136, 0.14);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-panel);
}

.hero-panel h2 {
  margin: 8px 0 0;
  color: var(--ink-strong);
  font-size: clamp(26px, 3vw, 36px);
  font-weight: 620;
  line-height: 1.12;
}

.hero-panel p {
  margin: 14px 0 0;
  color: var(--muted);
  font-size: 14px;
  line-height: 1.75;
}

.category-summary {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
  margin-top: 22px;
}

.category-summary-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  min-height: 52px;
  padding: 10px 12px;
  color: var(--ink);
  text-align: left;
  background: var(--paper-soft);
  border: 1px solid var(--line);
  border-radius: var(--radius-md);
  transition: background-color 120ms ease, border-color 120ms ease, transform 120ms ease;
}

.category-summary-item:hover,
.category-summary-item.is-active {
  background: var(--blue-soft);
  border-color: rgba(0, 56, 136, 0.32);
  transform: translateY(-1px);
}

.category-summary-item span {
  color: var(--muted);
  font-size: 13px;
  font-weight: 650;
}

.category-summary-item strong {
  color: var(--brand-blue);
  font-size: 20px;
}

```

```

    font-weight: 650;
}

.portal-divider {
  display: flex;
  gap: 8px;
  height: 6px;
  padding: 0 clamp(16px, 4vw, 56px);
  background: var(--divider-bg, transparent);
}

.portal-divider-line {
  flex: 1;
  height: 1px;
  margin: auto 0;
  background: linear-gradient(90deg, transparent, rgba(0, 56, 136, 0.32), rgba(0, 152, 56, 0.32),
transparent);
}

.systems-section {
  padding: 26px clamp(16px, 4vw, 56px) 48px;
  background: var(--canvas);
  border-top: 1px solid var(--systems-top-border, rgba(0, 56, 136, 0.12));
}

.catalog-heading {
  display: flex;
  align-items: end;
  justify-content: space-between;
  gap: 24px;
  margin-bottom: 18px;
}

.catalog-heading h2 {
  margin: 5px 0 0;
  color: var(--ink-strong);
  font-size: clamp(24px, 3vw, 36px);
  font-weight: 620;
  line-height: 1.18;
}

.catalog-heading p {
  max-width: 520px;
  margin: 0;
  color: var(--muted);
  font-size: 14px;
  line-height: 1.65;
}

.catalog-heading strong {
  color: var(--brand-blue);
}

.category-bar {
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 18px;
  padding: 8px;
  background: rgba(255, 255, 255, 0.62);
  border: 1px solid var(--line);
  border-radius: var(--radius-lg);
}

.category-pill {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  min-height: 36px;

```

```

padding: 6px 11px;
color: var(--muted);
background: transparent;
border: 1px solid transparent;
border-radius: var(--radius-sm);
font-size: 13px;
font-weight: 650;
}

.category-pill strong {
min-width: 24px;
padding: 2px 6px;
color: var(--brand-blue);
text-align: center;
background: #fff;
border: 1px solid var(--line);
border-radius: 999px;
font-size: 11px;
}

.category-pill:hover,
.category-pill.is-active {
color: var(--brand-blue);
background: var(--paper);
border-color: rgba(0, 56, 136, 0.22);
}

.category-pill.is-active {
box-shadow: inset 0 -2px 0 var(--brand-green);
}

.system-grid {
display: grid;
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 14px;
}

.system-card {
display: flex;
flex-direction: column;
min-height: 242px;
padding: 16px;
color: var(--ink);
background: var(--paper);
border: 1px solid var(--line);
border-radius: var(--radius-lg);
box-shadow: 0 1px 0 rgba(255, 255, 255, 0.9) inset;
transition: transform 120ms ease, border-color 120ms ease, box-shadow 120ms ease, background-col
or 120ms ease;
}

.system-card-link:hover {
transform: translateY(-2px);
border-color: rgba(0, 56, 136, 0.38);
box-shadow: var(--shadow-hover);
}

.system-card-link {
color: inherit;
text-decoration: none;
}

.system-card.is-placeholder {
background: linear-gradient(180deg, #fff, #fbfcfe);
}

.system-card-content {
display: flex;

```

```

    flex: 1;
    flex-direction: column;
    min-width: 0;
}

.system-card-layout {
    display: grid;
    grid-template-rows: 1fr auto;
    gap: 12px;
}

.system-card-top {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
    min-height: 26px;
}

.system-category {
    margin: 0;
    overflow: hidden;
    color: var(--brand-blue);
    font-size: 12px;
    font-weight: 700;
    line-height: 1.2;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.status-badge,
.tag-row span {
    display: inline-flex;
    align-items: center;
    min-height: 22px;
    border-radius: 999px;
    font-size: 11px;
    font-weight: 700;
}

.status-badge {
    flex: 0 0 auto;
    padding: 2px 8px;
    color: var(--brand-blue);
    background: var(--blue-soft);
    border: 1px solid rgba(0, 56, 136, 0.1);
}

.status-badge.is-ready {
    color: #04733a;
    background: var(--green-soft);
    border-color: rgba(0, 152, 56, 0.16);
}

.status-badge.is-maintenance {
    color: var(--warning);
    background: var(--warning-soft);
    border-color: rgba(154, 103, 0, 0.18);
}

.status-badge.is-pending {
    color: var(--muted);
    background: #f1f4f8;
    border-color: rgba(101, 115, 133, 0.16);
}

.system-card-body {
    flex: 1;

```

```

padding-top: 18px;
}

.system-title-line {
display: flex;
align-items: center;
gap: 11px;
}

.system-icon {
display: grid;
place-items: center;
flex: 0 0 auto;
width: 36px;
height: 36px;
color: var(--brand-blue);
background: var(--blue-soft);
border: 1px solid rgba(0, 56, 136, 0.12);
border-radius: var(--radius-sm);
}

.system-icon-svg {
width: 18px;
height: 18px;
fill: none;
stroke: currentColor;
stroke-linecap: round;
stroke-linejoin: round;
stroke-width: 1.8;
}

.system-card h2 {
margin: 0;
color: var(--ink-strong);
font-size: clamp(17px, 1.5vw, 20px);
font-weight: 650;
line-height: 1.28;
}

.system-card p {
margin: 12px 0 0;
color: var(--muted);
font-size: 13px;
line-height: 1.65;
}

.tag-row {
flex-wrap: wrap;
gap: 6px;
margin: 16px 0 0;
}

.tag-row span {
padding: 2px 8px;
color: var(--muted);
background: #f3f6f9;
border: 1px solid #e6edf3;
}

.system-card-footer {
display: flex;
align-items: center;
justify-content: space-between;
gap: 10px;
margin-top: 18px;
padding-top: 12px;
color: var(--muted);
border-top: 1px solid var(--line);
}

```

```

    font-size: 12px;
    font-weight: 650;
}

.system-card-link .system-card-footer {
    color: var(--brand-blue);
}

.system-card-media {
    min-height: 112px;
}

.system-card-image {
    width: 100%;
    aspect-ratio: 16 / 9;
    height: 100%;
    object-fit: cover;
    background: var(--blue-soft);
    border: 1px solid var(--line);
    border-radius: var(--radius-md);
}

.empty-state,
.loading-state {
    display: grid;
    place-items: center;
    min-height: 360px;
    padding: 40px 20px;
    text-align: center;
}

.empty-state {
    background: rgba(255, 255, 255, 0.72);
    border: 1px dashed var(--line-strong);
    border-radius: var(--radius-lg);
}

.empty-state-mark {
    display: grid;
    place-items: center;
    width: 64px;
    height: 64px;
    margin-bottom: 18px;
    color: var(--brand-blue);
    background: var(--blue-soft);
    border: 1px solid rgba(0, 56, 136, 0.12);
    border-radius: var(--radius-lg);
}

.empty-state h1,
.empty-state h2 {
    margin: 0;
    color: var(--ink-strong);
}

.empty-state p,
.loading-state p {
    max-width: 520px;
    margin: 12px 0 0;
    color: var(--muted);
    line-height: 1.65;
}

.load-error pre {
    overflow: auto;
    max-width: min(760px, 100%);
    padding: 14px;
    color: #fff;
}

```

```

    background: #111827;
    border-radius: var(--radius-md);
}

.site-footer {
    display: flex;
    justify-content: space-between;
    gap: 24px;
    padding: 24px clamp(16px, 4vw, 56px);
    color: var(--muted);
    background: #fff;
    border-top: 1px solid var(--line);
}

.site-footer strong {
    color: var(--ink-strong);
}

.site-footer p {
    margin: 6px 0 0;
    font-size: 12px;
    line-height: 1.55;
}

.site-footer a {
    color: var(--brand-blue);
    font-size: 13px;
    font-weight: 700;
}

.section-heading,
.mock-window,
.mock-dots,
.mock-line,
.mock-card-row {
    display: none;
}

.admin-root {
    background:
        linear-gradient(180deg, #ffffff 0, rgba(255, 255, 255, 0) 280px),
        var(--canvas);
}

.admin-login-shell {
    display: grid;
    min-height: 100vh;
    place-items: center;
    padding: 32px 18px;
}

.admin-login-card {
    width: min(520px, 100%);
    padding: clamp(24px, 4vw, 38px);
    background: rgba(255, 255, 255, 0.94);
    border: 1px solid var(--line);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-panel);
}

.admin-login-card .brand {
    margin-bottom: 32px;
}

.admin-login-form,
.admin-form {
    display: grid;
    gap: 16px;
}

```

```

}

.admin-login-form label,
.admin-form label,
.admin-search {
  display: grid;
  gap: 8px;
  color: var(--muted);
  font-size: 13px;
  font-weight: 700;
}

.admin-login-form input,
.admin-form input,
.admin-form select,
.admin-form textarea,
.admin-search input {
  width: 100%;
  min-height: 44px;
  padding: 10px 12px;
  color: var(--ink);
  background: #fff;
  border: 1px solid var(--line-strong);
  border-radius: var(--radius-sm);
  outline: 0;
  transition: border-color 120ms ease, box-shadow 120ms ease;
}

.admin-login-form input:focus,
.admin-form input:focus,
.admin-form select:focus,
.admin-form textarea:focus,
.admin-search input:focus {
  border-color: rgba(0, 56, 136, 0.48);
  box-shadow: 0 0 0 3px rgba(0, 152, 56, 0.1);
}

.admin-form textarea {
  min-height: 112px;
  resize: vertical;
}

.admin-upload {
  flex-wrap: wrap;
  gap: 10px;
  min-height: 48px;
  padding: 10px;
  background: #fff;
  border: 1px solid var(--line-strong);
  border-radius: var(--radius-sm);
}

.admin-upload-button {
  position: relative;
  overflow: hidden;
  min-height: 38px;
}

.admin-upload-button input[type="file"] {
  position: absolute;
  inset: 0;
  opacity: 0;
  cursor: pointer;
}

.admin-upload-path,
.admin-upload-help {
  color: var(--muted);
}

```



```

    font-size: 13px;
    line-height: 1.5;
}

.admin-upload-path {
    overflow: hidden;
    flex: 1 1 240px;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.admin-upload-remove,
.admin-danger {
    color: var(--danger);
    background: var(--danger-soft);
    border-color: #ffd1cc;
}

.admin-back-link {
    display: inline-flex;
    margin-top: 18px;
    color: var(--brand-blue);
    font-size: 14px;
    font-weight: 700;
}

.admin-hero {
    padding: clamp(34px, 5vw, 60px) clamp(16px, 4vw, 56px) 24px;
    background:
        linear-gradient(90deg, rgba(234, 241, 250, 0.9), rgba(255, 255, 255, 0.72), rgba(232, 246, 238,
0.7)),
        #fff;
    border-bottom: 1px solid var(--line);
}

.admin-hero h1 {
    max-width: 760px;
    margin: 6px 0 0;
    color: var(--ink-strong);
    font-size: clamp(32px, 5vw, 54px);
    font-weight: 650;
    line-height: 1.08;
}

.admin-hero p {
    max-width: 720px;
    margin: 14px 0 0;
    color: var(--muted);
    font-size: 16px;
    line-height: 1.7;
}

.admin-shell {
    display: grid;
    grid-template-columns: minmax(280px, 0.38fr) minmax(0, 1fr);
    gap: 18px;
    padding: 24px clamp(16px, 4vw, 56px) 56px;
}

.admin-alert {
    grid-column: 1 / -1;
    margin: 0;
    padding: 12px 14px;
    color: #075e34;
    background: var(--green-soft);
    border: 1px solid rgba(0, 152, 56, 0.18);
    border-radius: var(--radius-sm);
    font-weight: 700;
}

```

```

}

.admin-alert.is-error {
  color: var(--danger);
  background: var(--danger-soft);
  border-color: #ffd1cc;
}

.admin-list-panel,
.admin-form-panel,
.admin-preview {
  background: rgba(255, 255, 255, 0.9);
  border: 1px solid var(--line);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-panel);
}

.admin-list-panel,
.admin-form-panel {
  padding: 18px;
}

.admin-panel-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 16px;
}

.admin-panel-head h2 {
  margin: 5px 0 0;
  color: var(--ink-strong);
  font-size: 22px;
  font-weight: 650;
}

.admin-order-actions {
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 12px;
}

.admin-order-actions .button[disabled] {
  opacity: 0.45;
}

.admin-order-hint {
  color: var(--muted);
  font-size: 12px;
  line-height: 1.4;
}

.admin-card-list {
  display: grid;
  gap: 9px;
  margin-top: 14px;
}

.admin-system-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  width: 100%;
  min-height: 72px;
  padding: 12px;
  color: var(--ink);
}

```

```

    text-align: left;
    background: #fff;
    border: 1px solid var(--line);
    border-radius: var(--radius-md);
    transition: background-color 120ms ease, border-color 120ms ease, box-shadow 120ms ease;
}

.admin-system-row:hover,
.admin-system-row.is-active {
    background: var(--blue-soft);
    border-color: rgba(0, 56, 136, 0.35);
}

.admin-system-row.is-active {
    box-shadow: inset 3px 0 0 var(--brand-blue);
}

.admin-system-row strong,
.admin-system-row small {
    display: block;
}

.admin-system-row strong {
    color: var(--ink-strong);
    font-size: 14px;
    font-weight: 650;
}

.admin-system-row small,
.admin-system-row em {
    color: var(--muted);
    font-size: 12px;
    font-style: normal;
    line-height: 1.5;
}

.admin-form-panel {
    display: grid;
    align-content: start;
    grid-template-columns: minmax(0, 1fr) minmax(250px, 0.38fr);
    gap: 18px;
}

.admin-form-panel > .admin-panel-head {
    grid-column: 1 / -1;
}

.admin-form-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 14px;
}

.admin-form-actions {
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 4px;
}

.admin-preview {
    align-self: start;
    padding: 16px;
}

.admin-preview .system-card {
    min-height: 0;
    margin-top: 12px;
}

```

```

.admin-preview .system-icon {
  color: var(--brand-blue);
  font-size: 12px;
  font-weight: 750;
}

.admin-modal {
  position: fixed;
  inset: 0;
  z-index: 50;
  display: grid;
  place-items: center;
  padding: 24px;
  background: rgba(6, 20, 38, 0.28);
  backdrop-filter: blur(6px);
}

.admin-modal-card {
  width: min(420px, 100%);
  padding: 28px;
  background: rgba(255, 255, 255, 0.98);
  border: 1px solid var(--line);
  border-radius: var(--radius-lg);
  box-shadow: 0 28px 70px -40px rgba(0, 56, 136, 0.45);
}

.admin-modal-card h2 {
  margin: 0;
  color: var(--ink-strong);
  font-size: 26px;
  font-weight: 650;
}

.admin-modal-card p {
  margin: 12px 0 0;
  color: var(--muted);
  line-height: 1.65;
}

.admin-modal-actions {
  justify-content: center;
  margin-top: 20px;
}

html[data-theme="mintlify"],
html[data-theme="starbucks"] {
  --page-bg: var(--canvas);
  --hero-bottom-border: var(--line);
  --systems-top-border: var(--line);
  --divider-bg: transparent;
}

@media (max-width: 1080px) {
  .system-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .admin-shell,
  .admin-form-panel {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 920px) {
  .top-nav {
    align-items: flex-start;
    flex-direction: column;
  }
}

```

```

}

.nav-actions {
  justify-content: flex-start;
}

.hero-section {
  grid-template-columns: 1fr;
}

.hero-panel {
  max-width: 720px;
}

.catalog-heading {
  align-items: flex-start;
  flex-direction: column;
}
}

@media (max-width: 640px) {
  .top-nav {
    gap: 12px;
  }

  .brand strong {
    white-space: normal;
  }

  .nav-actions,
  .hero-actions,
  .button,
  .search-shell {
    width: 100%;
  }

  .metric-item {
    flex: 1 1 calc(50% - 10px);
  }

  .category-summary,
  .system-grid,
  .admin-form-grid {
    grid-template-columns: 1fr;
  }

  .category-bar {
    align-items: stretch;
    flex-direction: column;
  }

  .category-pill {
    justify-content: space-between;
    width: 100%;
  }

  .system-card {
    min-height: auto;
  }

  .admin-panel-head,
  .admin-system-row {
    align-items: stretch;
    flex-direction: column;
  }

  .site-footer {

```

```

        flex-direction: column;
    }
}

@media (prefers-reduced-motion: reduce) {
    *,
    *:before,
    *:after {
        scroll-behavior: auto !important;
        transition-duration: 0.01ms !important;
        animation-duration: 0.01ms !important;
    }
}

```

4.9 server.py

说明：Python 版后端服务。

```

#!/usr/bin/env python
# -*- coding: utf-8 -*-
from __future__ import print_function

try:
    import BaseHTTPServer
except ImportError:
    import http.server as BaseHTTPServer
try:
    import Cookie
except ImportError:
    import http.cookies as Cookie
import cgi
import errno
import hashlib
import json
import mimetypes
import os
import posixpath
import random
import re
import shutil
import tempfile
import time
try:
    from urllib import unquote
except ImportError:
    from urllib.parse import unquote

try:
    text_type = unicode
except NameError:
    text_type = str

ROOT_DIR = os.path.abspath(os.path.dirname(__file__))
CONFIG_PATH = os.path.join(ROOT_DIR, 'assets', 'config.json')
SESSION_COOKIE = 'portal_admin_session'
MAX_BODY_BYTES = 1024 * 1024
SESSIONS = {}

def ensure_dir(path):
    try:
        os.makedirs(path)
    except OSError as exc:
        if exc.errno != errno.EEXIST:
            raise

```

```

def json_response(handler, status, payload, headers=None):
    body = json.dumps(payload, ensure_ascii=False).encode('utf-8')
    handler.send_response(status)
    handler.send_header('Content-Type', 'application/json; charset=utf-8')
    handler.send_header('Cache-Control', 'no-store')
    if headers:
        for name, value in headers.items():
            handler.send_header(name, value)
    handler.send_header('Content-Length', str(len(body)))
    handler.end_headers()
    handler.wfile.write(body)

def text_response(handler, status, message):
    body = message.encode('utf-8')
    handler.send_response(status)
    handler.send_header('Content-Type', 'text/plain; charset=utf-8')
    handler.send_header('Content-Length', str(len(body)))
    handler.end_headers()
    handler.wfile.write(body)

def read_body(handler):
    length = int(handler.headers.get('Content-Length') or '0')
    if length > MAX_BODY_BYTES:
        raise ValueError('Request body is too large')
    raw = handler.rfile.read(length) if length else '{}'
    try:
        return json.loads(raw.decode('utf-8'))
    except Exception:
        raise ValueError('Request body must be valid JSON')

def read_upload(handler):
    form = cgi.FieldStorage(
        fp=handler.rfile,
        headers=handler.headers,
        environ={
            'REQUEST_METHOD': 'POST',
            'CONTENT_TYPE': handler.headers.get('Content-Type', '')
        }
    )
    image = form['image'] if 'image' in form else None
    if image is None or not getattr(image, 'filename', None):
        raise ValueError('Image file is required')
    content_type = getattr(image, 'type', '') or 'application/octet-stream'
    extension = os.path.splitext(image.filename)[1].lower()
    allowed = {
        'image/jpeg': '.jpg',
        'image/png': '.png',
        'image/webp': '.webp',
        'image/gif': '.gif'
    }
    if content_type in allowed:
        extension = allowed[content_type]
    if extension not in ['.jpg', '.jpeg', '.png', '.webp', '.gif']:
        raise ValueError('Only jpg, png, webp, and gif images are supported')
    return image.file.read(), extension

def read_config():
    with open(CONFIG_PATH, 'rb') as handle:
        return json.loads(handle.read().decode('utf-8'))

def write_config(config):
    backup_dir = os.path.join(ROOT_DIR, 'data', 'backups')
    ensure_dir(backup_dir)

```

```

stamp = time.strftime('%Y%m%d-%H%M%S')
shutil.copy2(CONFIG_PATH, os.path.join(backup_dir, 'config-%s.json' % stamp))
fd, temp_path = tempfile.mkstemp(prefix='config-', suffix='.json', dir=os.path.dirname(CONFIG_PATH))
try:
    with os.fdopen(fd, 'wb') as handle:
        handle.write((json.dumps(config, ensure_ascii=False, indent=2) + '\n').encode('utf-8'))
    os.rename(temp_path, CONFIG_PATH)
finally:
    if os.path.exists(temp_path):
        os.remove(temp_path)

def write_upload(content, extension):
    upload_dir = os.path.join(ROOT_DIR, 'assets', 'uploads')
    ensure_dir(upload_dir)
    filename = '%s-%s%s' % (
        time.strftime('%Y%m%d%H%M%S'),
        hashlib.sha1(os.urandom(16)).hexdigest()[:8],
        '.jpg' if extension == '.jpeg' else extension
    )
    file_path = os.path.join(upload_dir, filename)
    with open(file_path, 'wb') as handle:
        handle.write(content)
    return 'assets/uploads/%s' % filename

def delete_upload(upload_path):
    normalized = unicode_or_string(upload_path).strip()
    if not re.match(r'^assets/uploads/[a-z0-9-]+\.(jpg|jpeg|png|webp|gif)$', normalized, re.I):
        raise ValueError('Upload path is invalid')
    target = os.path.abspath(os.path.join(ROOT_DIR, normalized.replace('/', os.sep)))
    upload_root = os.path.abspath(os.path.join(ROOT_DIR, 'assets', 'uploads'))
    if target != upload_root and not target.startswith(upload_root + os.sep):
        raise ValueError('Upload path is invalid')
    if not os.path.exists(target):
        raise ValueError('Upload path is invalid')
    os.remove(target)

def parse_cookie(header):
    cookie = Cookie.SimpleCookie()
    if header:
        cookie.load(header)
    return cookie

def is_authenticated(handler):
    cookie = parse_cookie(handler.headers.get('Cookie'))
    if SESSION_COOKIE not in cookie:
        return False
    return cookie[SESSION_COOKIE].value in SESSIONS

def require_auth(handler):
    if is_authenticated(handler):
        return True
    json_response(handler, 401, {'error': 'Authentication required'})
    return False

def session_id():
    seed = '%s:%s:%s' % (time.time(), random.random(), os.urandom(32))
    return hashlib.sha256(seed).hexdigest()

def normalize_tags(value):

```



```

        if isinstance(value, list):
            return [unicode_or_string(item).strip() for item in value if unicode_or_string(item).strip()][:8]
        return [part.strip() for part in unicode_or_string(value).split(',') if part.strip()][:8]

def unicode_or_string(value):
    if value is None:
        return u''
    if isinstance(value, text_type):
        return value
    if isinstance(value, bytes):
        return value.decode('utf-8')
    return text_type(value)

def decode_url_component(value):
    decoded = unquote(value)
    if isinstance(decoded, text_type):
        return decoded
    return decoded.decode('utf-8')

def make_slug(value):
    text = unicode_or_string(value).strip().lower()
    text = re.sub(u'^[a-z0-9\u4e00-\u9fa5]+', '-', text, flags=re.UNICODE).strip('-')
    if text:
        return text
    return 'system-%s' % hashlib.sha1(str(time.time()).encode('utf-8')).hexdigest()[:8]

def validate_url(value, field):
    text = unicode_or_string(value).strip()
    if not text:
        return ''
    if text == '#':
        return text
    if text == 'image' and re.search(r'(^|[/\])\.\.([/\\]|$)', text):
        raise ValueError('Image path cannot contain parent-directory traversal')
    if re.match(r'^((https?:)?//', text, re.I) or re.match(r'^[a-z0-9_./#?=&~]+$', text, re.I):
        return text
    raise ValueError('%s is not a supported URL or relative path' % field)

def normalize_system(payload, existing_id=None):
    name = unicode_or_string(payload.get('name')).strip()
    description = unicode_or_string(payload.get('description')).strip()
    category = unicode_or_string(payload.get('category')).strip()
    if not name:
        raise ValueError('System name is required')
    if not description:
        raise ValueError('Description is required')
    if not category:
        raise ValueError('Category is required')
    return {
        'id': existing_id or make_slug(name),
        'name': name,
        'description': description,
        'category': category,
        'url': validate_url(payload.get('url') or '#', 'url'),
        'status': unicode_or_string(payload.get('status') or '待配置').strip(),
        'icon': unicode_or_string(payload.get('icon') or 'layout-dashboard').strip(),
        'image': validate_url(payload.get('image') or '', 'image'),
        'tags': normalize_tags(payload.get('tags'))
    }

```

```

class PortalHandler(BaseHTTPServer.BaseHTTPRequestHandler):
    server_version = 'ServicePortal/1.0'

    def do_GET(self):
        if self.path.startswith('/api/'):
            self.handle_api()
        else:
            self.serve_static()

    def do_POST(self):
        self.handle_api()

    def do_PUT(self):
        self.handle_api()

    def do_DELETE(self):
        self.handle_api()

    def handle_api(self):
        path = self.path.split('?', 1)[0]
        try:
            if path == '/api/session' and self.command == 'GET':
                json_response(self, 200, {'authenticated': is_authenticated(self)})
                return

            if path == '/api/login' and self.command == 'POST':
                password = os.environ.get('ADMIN_PASSWORD', '')
                if not password:
                    json_response(self, 503, {'error': 'Admin password is not configured'})
                    return
                body = read_body(self)
                if unicode_or_string(body.get('password')) != password:
                    json_response(self, 401, {'error': 'Invalid password'})
                    return
                sid = session_id()
                SESSIONS[sid] = time.time()
                json_response(self, 200, {'authenticated': True}, {
                    'Set-Cookie': '%s=%s; Path=/; HttpOnly; SameSite=Strict' % (SESSION_COOKIE, si
d)
                })
                return

            if path == '/api/logout' and self.command == 'POST':
                cookie = parse_cookie(self.headers.get('Cookie'))
                if SESSION_COOKIE in cookie:
                    SESSIONS.pop(cookie[SESSION_COOKIE].value, None)
                json_response(self, 200, {'authenticated': False}, {
                    'Set-Cookie': '%s=; Path=/; Max-Age=0; HttpOnly; SameSite=Strict' % SESSION_CO
OKIE
                })
                return

            if path == '/api/config' and self.command == 'GET':
                if not require_auth(self):
                    return
                json_response(self, 200, read_config())
                return

            if path == '/api/config' and self.command == 'PUT':
                if not require_auth(self):
                    return
                body = read_body(self)
                config = read_config()
                systems = body.get('systems')
                if not isinstance(systems, list):
                    json_response(self, 400, {'error': 'Systems array is required'})
                    return
                existing_ids = set(item.get('id') for item in config.get('systems', []))

```

```

        next_ids = set(item.get('id') for item in systems)
        if len(existing_ids) != len(next_ids) or any(system_id not in next_ids for system_
id in existing_ids):
            json_response(self, 400, {'error': 'Systems payload must contain the same ids
'})

        return
        config['systems'] = systems
        write_config(config)
        json_response(self, 200, {'config': config})
        return

if path == '/api/upload' and self.command == 'POST':
    if not require_auth(self):
        return
    content, extension = read_upload(self)
    saved_path = write_upload(content, extension)
    json_response(self, 201, {'path': saved_path})
    return

if path == '/api/upload' and self.command == 'DELETE':
    if not require_auth(self):
        return
    body = read_body(self)
    delete_upload(body.get('path'))
    json_response(self, 200, {'removed': body.get('path')})
    return

if path == '/api/systems' and self.command == 'POST':
    if not require_auth(self):
        return
    config = read_config()
    system = normalize_system(read_body(self))
    ids = set(item.get('id') for item in config.get('systems', []))
    base_id = system['id']
    counter = 2
    while system['id'] in ids:
        system['id'] = '%s-%s' % (base_id, counter)
        counter += 1
    config['systems'].append(system)
    write_config(config)
    json_response(self, 201, {'system': system, 'config': config})
    return

match = re.match(r'^/api/systems/([^\s/]+)$', path)
if match and self.command == 'PUT':
    if not require_auth(self):
        return
    system_id = decode_url_component(match.group(1))
    config = read_config()
    systems = config.get('systems', [])
    for index, item in enumerate(systems):
        if item.get('id') == system_id:
            system = normalize_system(read_body(self), system_id)
            systems[index] = system
            write_config(config)
            json_response(self, 200, {'system': system, 'config': config})
            return
    json_response(self, 404, {'error': 'System not found'})
    return

if match and self.command == 'DELETE':
    if not require_auth(self):
        return
    system_id = decode_url_component(match.group(1))
    config = read_config()
    next_systems = [item for item in config.get('systems', []) if item.get('id') != sy
stem_id]

    if len(next_systems) == len(config.get('systems', [])):

```

```

        json_response(self, 404, {'error': 'System not found'})
        return
    config['systems'] = next_systems
    write_config(config)
    json_response(self, 200, {'removed': system_id, 'config': config})
    return

    json_response(self, 404, {'error': 'API route not found'})
except ValueError as exc:
    json_response(self, 400, {'error': unicode_or_string(exc)})
except Exception as exc:
    json_response(self, 500, {'error': unicode_or_string(exc)})

def serve_static(self):
    request_path = self.path.split('?', 1)[0]
    request_path = '/index.html' if request_path == '/' else request_path
    request_path = posixpath.normpath(decode_url_component(request_path)).lstrip('/')
    file_path = os.path.abspath(os.path.join(ROOT_DIR, request_path))
    if not file_path.startswith(ROOT_DIR + os.sep):
        text_response(self, 403, 'Forbidden')
        return
    if not os.path.isfile(file_path):
        text_response(self, 404, 'Not found')
        return
    content_type = mimetypes.guess_type(file_path)[0] or 'application/octet-stream'
    self.send_response(200)
    self.send_header('Content-Type', content_type)
    self.send_header('Content-Length', str(os.path.getsize(file_path)))
    self.end_headers()
    with open(file_path, 'rb') as handle:
        shutil.copyfileobj(handle, self.wfile)

if __name__ == '__main__':
    port = int(os.environ.get('PORT', '80'))
    server = BaseHTTPServer.HTTPServer(('0.0.0.0', port), PortalHandler)
    print('Service portal listening on %s' % port)
    server.serve_forever()

```

4.10 server.js

说明：Node.js 版后端服务。

```

const http = require('node:http');
const { createReadStream } = require('node:fs');
const { access, copyFile, mkdir, readFile, rename, writeFile } = require('node:fs/promises');
const path = require('node:path');
const crypto = require('node:crypto');

const SESSION_COOKIE = 'portal_admin_session';
const MAX_BODY_BYTES = 1024 * 1024;

const contentTypeTypes = {
    '.html': 'text/html; charset=utf-8',
    '.js': 'text/javascript; charset=utf-8',
    '.css': 'text/css; charset=utf-8',
    '.json': 'application/json; charset=utf-8',
    '.jpg': 'image/jpeg',
    '.jpeg': 'image/jpeg',
    '.png': 'image/png',
    '.svg': 'image/svg+xml',
    '.ico': 'image/x-icon'
};

function jsonResponse(response, status, payload, headers = {}) {
    response.writeHead(status, {
        'content-type': 'application/json; charset=utf-8',

```

```

        'cache-control': 'no-store',
        ...headers
    });
    response.end(JSON.stringify(payload));
}

function textResponse(response, status, message) {
    response.writeHead(status, {
        'content-type': 'text/plain; charset=utf-8',
        'cache-control': 'no-store'
    });
    response.end(message);
}

function parseCookies(header = '') {
    return header.split(';').reduce((cookies, part) => {
        const [rawName, ...rawValue] = part.trim().split('=');
        if (!rawName) return cookies;
        cookies[rawName] = decodeURIComponent(rawValue.join('='));
        return cookies;
    }, {});
}

function readRequestBody(request) {
    return new Promise((resolve, reject) => {
        let size = 0;
        const chunks = [];
        request.on('data', (chunk) => {
            size += chunk.length;
            if (size > MAX_BODY_BYTES) {
                reject(new Error('Request body is too large'));
                request.destroy();
                return;
            }
            chunks.push(chunk);
        });
        request.on('end', () => {
            const raw = Buffer.concat(chunks).toString('utf8') || '{}';
            try {
                resolve(JSON.parse(raw));
            } catch {
                reject(new Error('Request body must be valid JSON'));
            }
        });
        request.on('error', reject);
    });
}

function parseMultipart(request) {
    return new Promise((resolve, reject) => {
        const header = request.headers['content-type'] || '';
        const match = header.match(/boundary=(?:"([^\"]+)"|([^\;]+))/i);
        if (!match) {
            reject(new Error('Multipart boundary is required'));
            return;
        }
        const boundary = Buffer.from(`--${match[1] || match[2]}`);
        let size = 0;
        const chunks = [];
        request.on('data', (chunk) => {
            size += chunk.length;
            if (size > MAX_BODY_BYTES * 8) {
                reject(new Error('Upload body is too large'));
                request.destroy();
                return;
            }
            chunks.push(chunk);
        });
    });
}

```

```

request.on('end', () => {
  const body = Buffer.concat(chunks);
  const parts = [];
  let start = body.indexOf(boundary);
  while (start !== -1) {
    const next = body.indexOf(boundary, start + boundary.length);
    if (next === -1) break;
    const part = body.subarray(start + boundary.length + 2, next - 2);
    start = next;
    if (!part.length) continue;
    const headerEnd = part.indexOf(Buffer.from('\r\n\r\n'));
    if (headerEnd === -1) continue;
    const rawHeaders = part.subarray(0, headerEnd).toString('utf8');
    const content = part.subarray(headerEnd + 4);
    const disposition = rawHeaders.match(/name="([^"]+)"/i);
    const filename = rawHeaders.match(/filename="([^"]*)"/i);
    const contentType = rawHeaders.match(/content-type:\s*([^\\r\\n]+)/i);
    parts.push({
      name: disposition ? disposition[1] : '',
      filename: filename ? filename[1] : '',
      contentType: contentType ? contentType[1].trim() : 'application/octet-stream',
      content
    });
  }
  resolve(parts);
});
request.on('error', reject);
});
}

function safeJoin(rootDir, requestPath) {
  const normalized = path.normalize(decodeURIComponent(requestPath)).replace(/^(\\.\\.\\/\\)/, '');
  const resolved = path.resolve(rootDir, normalized.replace(/\\/\\/, '/'));
  const root = path.resolve(rootDir);
  if (resolved !== root && !resolved.startsWith(root + path.sep)) {
    return null;
  }
  return resolved;
}

function makeSlug(value) {
  const slug = String(value || '')
    .trim()
    .toLowerCase()
    .replace(/^[a-z0-9\u4e00-\u9fa5]+/g, '-')
    .replace(/^-+|-+$/g, '');
  return slug || `system-${crypto.randomBytes(4).toString('hex')}`;
}

function normalizeTags(value) {
  if (Array.isArray(value)) {
    return value.map((tag) => String(tag).trim()).filter(Boolean).slice(0, 8);
  }
  return String(value || '')
    .split(',')
    .map((tag) => tag.trim())
    .filter(Boolean)
    .slice(0, 8);
}

function validateUrl(value, field) {
  const text = String(value || '').trim();
  if (!text) return '#';
  if (text === '#') return text;
  if (field === 'image' && /^(^[\\\\/]\.\.\/([\\\\/])$)/.test(text)) {
    throw new Error('Image path cannot contain parent-directory traversal');
  }
  if (/^(https?:)?\:\/\/.i.test(text) || /^[a-z0-9_./#?=&-%]+$/i.test(text)) {

```

```

    return text;
  }
  throw new Error(`${field} is not a supported URL or relative path`);
}

function normalizeSystem(input, existingId) {
  const name = String(input.name || '').trim();
  const description = String(input.description || '').trim();
  const category = String(input.category || '').trim();
  const icon = String(input.icon || '').trim() || 'layout-dashboard';
  const status = String(input.status || '').trim() || '待配置';

  if (!name) throw new Error('System name is required');
  if (!description) throw new Error('Description is required');
  if (!category) throw new Error('Category is required');

  return {
    id: existingId || makeSlug(name),
    name,
    description,
    category,
    url: validateUrl(input.url, 'url'),
    status,
    icon,
    image: validateUrl(input.image || '', 'image'),
    tags: normalizeTags(input.tags)
  };
}

async function readConfig(configPath) {
  return JSON.parse(await readFile(configPath, 'utf8'));
}

async function writeConfig(rootDir, configPath, config) {
  const backupDir = path.join(rootDir, 'data', 'backups');
  await mkdir(backupDir, { recursive: true });
  const stamp = new Date().toISOString().replace(/[:.]/g, '-');
  await copyFile(configPath, path.join(backupDir, `config-${stamp}.json`));
  const tempPath = `${configPath}.${process.pid}.${Date.now()}.tmp`;
  await writeFile(tempPath, `${JSON.stringify(config, null, 2)}\n`, 'utf8');
  await rename(tempPath, configPath);
}

async function storeUpload(rootDir, part) {
  const allowed = new Map([
    ['image/jpeg', '.jpg'],
    ['image/png', '.png'],
    ['image/webp', '.webp'],
    ['image/gif', '.gif']
  ]);
  const extension = allowed.get(part.contentType) || path.extname(part.filename || '').toLowerCase();
  if (!['.jpg', '.jpeg', '.png', '.webp', '.gif'].includes(extension)) {
    throw new Error('Only jpg, png, webp, and gif images are supported');
  }
  const uploadDir = path.join(rootDir, 'assets', 'uploads');
  await mkdir(uploadDir, { recursive: true });
  const filename = `${Date.now().toString(36)}-${crypto.randomBytes(4).toString('hex')}${extension}`;
  await writeFile(path.join(uploadDir, filename), part.content);
  return `assets/uploads/${filename}`;
}

async function deleteUpload(rootDir, uploadPath) {
  const normalized = String(uploadPath || '').trim();
  if (!/^assets\/uploads\/[a-z0-9]+\.(jpg|jpeg|png|webp|gif)$/i.test(normalized)) {
    throw new Error('Upload path is invalid');
  }
}

```

```

const target = safeJoin(rootDir, normalized);
if (!target) {
  throw new Error('Upload path is invalid');
}
await access(target);
const { unlink } = await import('node:fs/promises');
await unlink(target);
}

function isAuthenticated(request, sessions) {
  const cookies = parseCookies(request.headers.cookie);
  const session = cookies[SESSION_COOKIE];
  return Boolean(session && sessions.has(session));
}

function requireAuth(request, response, sessions) {
  if (isAuthenticated(request, sessions)) return true;
  jsonResponse(response, 401, { error: 'Authentication required' });
  return false;
}

async function serveStatic(rootDir, requestPath, response) {
  const pathname = requestPath === '/' ? '/index.html' : requestPath;
  const filePath = safeJoin(rootDir, pathname);
  if (!filePath) {
    textResponse(response, 403, 'Forbidden');
    return;
  }

  try {
    await access(filePath);
  } catch {
    textResponse(response, 404, 'Not found');
    return;
  }

  response.writeHead(200, {
    'content-type': contentType(path.extname(filePath).toLowerCase()) || 'application/octet-stream'
  });
  createReadStream(filePath).pipe(response);
}

function createServer(options = {}) {
  const rootDir = path.resolve(options.rootDir || process.cwd());
  const configPath = path.join(rootDir, 'assets', 'config.json');
  const adminPassword = options.adminPassword || process.env.ADMIN_PASSWORD || '';
  const sessions = new Map();

  return http.createServer(async (request, response) => {
    const url = new URL(request.url, 'http://localhost');

    try {
      if (url.pathname === '/api/session' && request.method === 'GET') {
        jsonResponse(response, 200, { authenticated: isAuthenticated(request, sessions) });
        return;
      }

      if (url.pathname === '/api/login' && request.method === 'POST') {
        if (!adminPassword) {
          jsonResponse(response, 503, { error: 'Admin password is not configured' });
          return;
        }
        const body = await readRequestBody(request);
        if (String(body.password || '') !== adminPassword) {
          jsonResponse(response, 401, { error: 'Invalid password' });
          return;
        }
      }
    }
  });
}

```



```

    const sessionId = crypto.randomBytes(32).toString('base64url');
    sessions.set(sessionId, { createdAt: Date.now() });
    jsonResponse(response, 200, { authenticated: true }, {
      'set-cookie': `${SESSION_COOKIE}=${encodeURIComponent(sessionId)}; Path=/; HttpOnly; SameSite=Strict`
    });
    return;
  }

  if (url.pathname === '/api/logout' && request.method === 'POST') {
    const cookies = parseCookies(request.headers.cookie);
    if (cookies[SESSION_COOKIE]) sessions.delete(cookies[SESSION_COOKIE]);
    jsonResponse(response, 200, { authenticated: false }, {
      'set-cookie': `${SESSION_COOKIE}=${encodeURIComponent(sessionId)}; Path=/; Max-Age=0; HttpOnly; SameSite=Strict`
    });
    return;
  }

  if (url.pathname === '/api/config' && request.method === 'GET') {
    if (!requireAuth(request, response, sessions)) return;
    jsonResponse(response, 200, await readConfig(configPath));
    return;
  }

  if (url.pathname === '/api/config' && request.method === 'PUT') {
    if (!requireAuth(request, response, sessions)) return;
    const body = await readRequestBody(request);
    const config = await readConfig(configPath);
    if (!Array.isArray(body.systems)) {
      jsonResponse(response, 400, { error: 'Systems array is required' });
      return;
    }
    const existingIds = new Set(config.systems.map((item) => item.id));
    const nextIds = new Set(body.systems.map((item) => item.id));
    if (existingIds.size !== nextIds.size || [...existingIds].some((id) => !nextIds.has(id))) {
      jsonResponse(response, 400, { error: 'Systems payload must contain the same ids' });
      return;
    }
    config.systems = body.systems;
    await writeConfig(rootDir, configPath, config);
    jsonResponse(response, 200, { config });
    return;
  }

  if (url.pathname === '/api/upload' && request.method === 'POST') {
    if (!requireAuth(request, response, sessions)) return;
    const parts = await parseMultipart(request);
    const image = parts.find((part) => part.name === 'image' && part.filename);
    if (!image) {
      jsonResponse(response, 400, { error: 'Image file is required' });
      return;
    }
    const savedPath = await storeUpload(rootDir, image);
    jsonResponse(response, 201, { path: savedPath });
    return;
  }

  if (url.pathname === '/api/upload' && request.method === 'DELETE') {
    if (!requireAuth(request, response, sessions)) return;
    const body = await readRequestBody(request);
    await deleteUpload(rootDir, body.path);
    jsonResponse(response, 200, { removed: body.path });
    return;
  }

  if (url.pathname === '/api/systems' && request.method === 'POST') {
    if (!requireAuth(request, response, sessions)) return;

```

```

    const body = await readRequestBody(request);
    const config = await readConfig(configPath);
    const system = normalizeSystem(body);
    const ids = new Set(config.systems.map((item) => item.id));
    let baseId = system.id;
    let counter = 2;
    while (ids.has(system.id)) {
        system.id = `${baseId}-${counter}`;
        counter += 1;
    }
    config.systems.push(system);
    await writeConfig(rootDir, configPath, config);
    jsonResponse(response, 201, { system, config });
    return;
}

const systemMatch = url.pathname.match(/^\/api\/systems\/([^\s]+)$/);
if (systemMatch && request.method === 'PUT') {
    if (!requireAuth(request, response, sessions)) return;
    const id = decodeURIComponent(systemMatch[1]);
    const body = await readRequestBody(request);
    const config = await readConfig(configPath);
    const index = config.systems.findIndex((item) => item.id === id);
    if (index === -1) {
        jsonResponse(response, 404, { error: 'System not found' });
        return;
    }
    const system = normalizeSystem(body, id);
    config.systems[index] = system;
    await writeConfig(rootDir, configPath, config);
    jsonResponse(response, 200, { system, config });
    return;
}

if (systemMatch && request.method === 'DELETE') {
    if (!requireAuth(request, response, sessions)) return;
    const id = decodeURIComponent(systemMatch[1]);
    const config = await readConfig(configPath);
    const nextSystems = config.systems.filter((item) => item.id !== id);
    if (nextSystems.length === config.systems.length) {
        jsonResponse(response, 404, { error: 'System not found' });
        return;
    }
    config.systems = nextSystems;
    await writeConfig(rootDir, configPath, config);
    jsonResponse(response, 200, { removed: id, config });
    return;
}

if (url.pathname.startsWith('/api/')) {
    jsonResponse(response, 404, { error: 'API route not found' });
    return;
}

if (request.method !== 'GET' && request.method !== 'HEAD') {
    textResponse(response, 405, 'Method not allowed');
    return;
}

await serveStatic(rootDir, url.pathname, response);
} catch (error) {
    const status = /required|valid|supported|traversal/i.test(error.message) ? 400 : 500;
    jsonResponse(response, status, { error: error.message });
}
});
}

exports.createServer = createServer;

```

```
if (require.main === module) {  
  const port = Number(process.env.PORT || 80);  
  createServer().listen(port, '0.0.0.0', () => {  
    console.log(`Service portal listening on ${port}`);  
  });  
}
```

5. 结论

以上源代码构成自动化系统定制化交互软件 V1.0 的主要程序内容，能够实现门户展示、入口搜索筛选、后台认证、配置维护、图片上传和配置备份等功能。